

PRACTICAL WEEK 5

Task-1

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
srikar@DESKTOP-0NIMS2G: ~/week4/week5
srikar@DESKTOP-0NIMS2G:~/week4$ mkdir week5 && cd week5
srikar@DESKTOP-0NIMS2G:~/week4/week5$ nano producer_consumer.c
srikar@DESKTOP-0NIMS2G:~/week4/week5$ gcc producer_consumer.c -o producer_consumer
srikar@DESKTOP-0NIMS2G:~/week4/week5$ ./producer_consumer
Child (Consumer): Received 1000 integers
Parent (Producer): Sent 1000 integers
Data transferred: 4000 bytes
Communication time: 0.000003799 seconds
Throughput: 1052908660.17 bytes/second
srikar@DESKTOP-0NIMS2G:~/week4/week5$
```

Observation

- An anonymous pipe was successfully created for communication between the parent and child processes.
- The parent process acted as the producer and generated 1000 integer values.
- The child process acted as the consumer and read the data from the pipe.

- The parent and child communicated successfully without using a shared file.
- The communication time was measured using `clock_gettime()`.
- Communication throughput was calculated in bytes per second, providing a measure of communication efficiency.
- The parent used `wait()` to synchronize with the child and ensure that the consumer completed before the parent terminated.
- Thus, anonymous pipes provide a simple and efficient one-way inter-process communication (IPC) mechanism between related processes.

Task -2 Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    // Create the pipe
    if (pipe(pipefd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    // Create first child for 'ls -l'
    pid1 = fork();
    if (pid1 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid1 == 0) {
        // Child 1: Redirect stdout to write end of pipe
        close(pipefd[0]); // Close read end
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls failed");
        exit(1);
    }

    // Create second child for 'grep .c'
    pid2 = fork();
    if (pid2 < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid2 == 0) {
        // Child 2: Redirect stdin from read end of pipe
        close(pipefd[1]); // Close write end
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);

        execlp("grep", "grep", ".c", NULL);
        perror("execlp grep failed");
        exit(1);
    }

    // Parent process closes both ends of pipe
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both children to terminate
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline execution completed.\n");
}

```



srikar@DESKTOP-0NIMS2G: ~/week6

```
srikar@DESKTOP-0NIMS2G:~$ cd ~/week6
```

```
srikar@DESKTOP-0NIMS2G:~/week6$ nano client.c
```

```
srikar@DESKTOP-0NIMS2G:~/week6$ gcc client.c -o client
```

```
srikar@DESKTOP-0NIMS2G:~/week6$ ./client
```

Client connected to server.

Enter message:

Server: Server processed:

Enter message:

Server: Server processed:

Enter message: Hello Server

Server: Server processed: Hello Server

Enter message: How are you?

Server: Server processed: How are you?

Enter message: exit

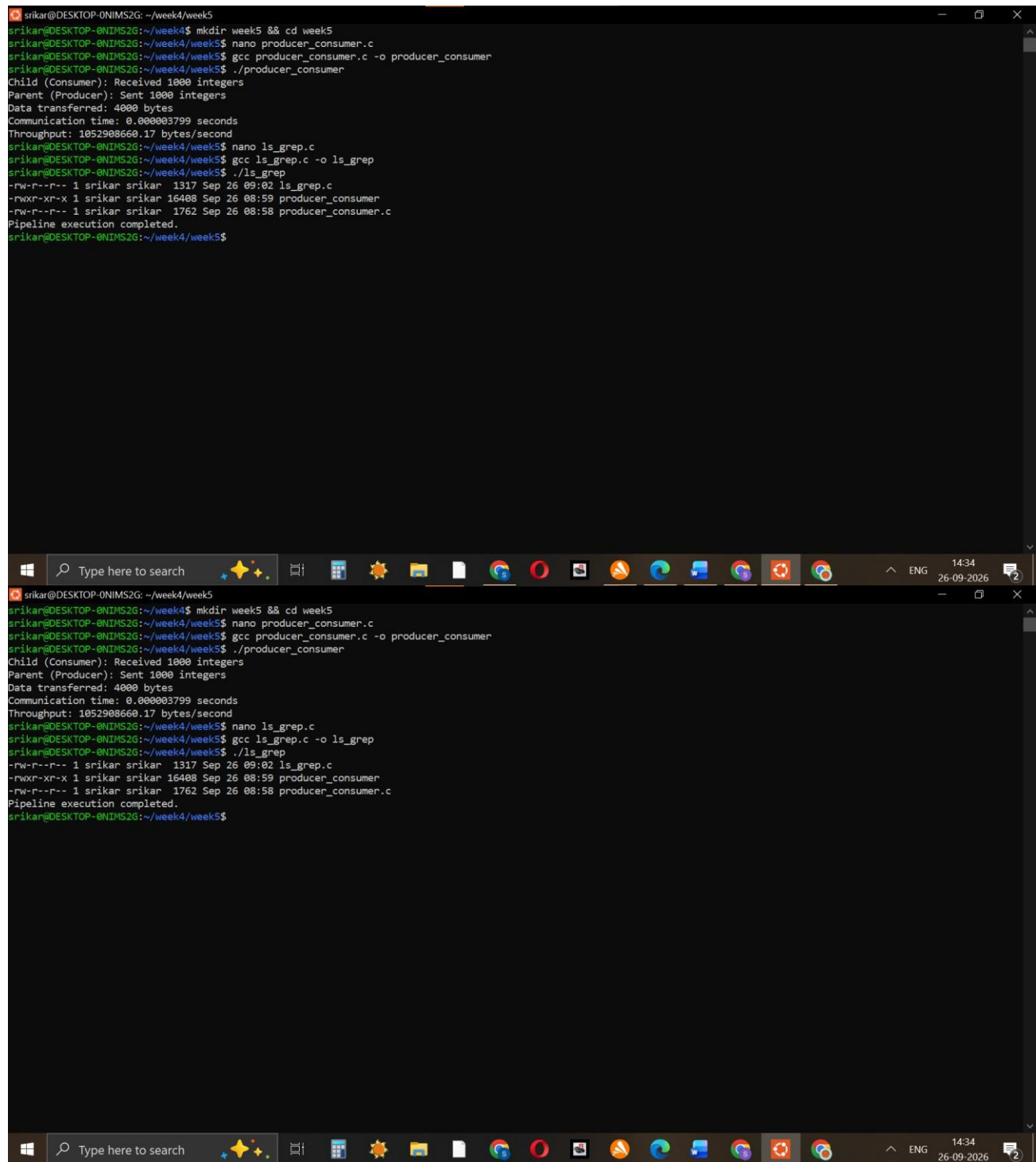
Server: Server processed: exit

```
srikar@DESKTOP-0NIMS2G:~/week6$
```

```
srikan@DESKTOP-0NIMS2G: ~/week6
srikan@DESKTOP-0NIMS2G:~$ cd ~/week6
srikan@DESKTOP-0NIMS2G:~/week6$ nano client.c
srikan@DESKTOP-0NIMS2G:~/week6$ gcc client.c -o client
srikan@DESKTOP-0NIMS2G:~/week6$ ./client
Client connected to server.
Enter message:
Server: Server processed:
Enter message:
Server: Server processed:
Enter message: Hello Server
Server: Server processed: Hello Server
Enter message: How are you?
Server: Server processed: How are you?
Enter message: exit
Server: Server processed: exit
srikan@DESKTOP-0NIMS2G:~/week6$ Server started...
ing for client message...Server: command not found
srikan@DESKTOP-0NIMS2G:~/week6$ Waiting for client message...cd ~/week6
Waiting: command not found
srikan@DESKTOP-0NIMS2G:~/week6$ cd ~/week6
srikan@DESKTOP-0NIMS2G:~/week6$ ./server
Server started...
Waiting for client message...
Client: Client 1 message
Response sent to client.
Waiting for client message...
Client: ./client
Response sent to client.
Waiting for client message...
Client: Client 2 message
Response sent to client.
Waiting for client message...

srikan@DESKTOP-0NIMS2G: ~/week4/week5
srikan@DESKTOP-0NIMS2G:~/week4$ mkdir week5 && cd week5
srikan@DESKTOP-0NIMS2G:~/week4/week5$ nano producer_consumer.c
srikan@DESKTOP-0NIMS2G:~/week4/week5$ gcc producer_consumer.c -o producer_consumer
srikan@DESKTOP-0NIMS2G:~/week4/week5$ ./producer_consumer
Child (Consumer): Received 1000 integers
Parent (Producer): Sent 1000 integers
Data transferred: 4000 bytes
Communication time: 0.00003799 seconds
Throughput: 1052908660.17 bytes/second
srikan@DESKTOP-0NIMS2G:~/week4/week5$ nano ls_grep.c
srikan@DESKTOP-0NIMS2G:~/week4/week5$ gcc ls_grep.c -o ls_grep
srikan@DESKTOP-0NIMS2G:~/week4/week5$ ./ls_grep
-rw-r--r-- 1 srikan srikan 1317 Sep 26 09:02 ls_grep.c
-rwxr-xr-x 1 srikan srikan 16408 Sep 26 08:59 producer_consumer
-rw-r--r-- 1 srikan srikan 1762 Sep 26 08:58 producer_consumer.c
Pipeline execution completed.
srikan@DESKTOP-0NIMS2G:~/week4/week5$
```

```
srikan@DESKTOP-0NIMS2G: ~/week4/week5
srikan@DESKTOP-0NIMS2G:~/week4$ mkdir week5 && cd week5
srikan@DESKTOP-0NIMS2G:~/week4/week5$ nano producer_consumer.c
srikan@DESKTOP-0NIMS2G:~/week4/week5$ gcc producer_consumer.c -o producer_consumer
srikan@DESKTOP-0NIMS2G:~/week4/week5$ ./producer_consumer
Child (Consumer): Received 1000 integers
Parent (Producer): Sent 1000 integers
Data transferred: 4000 bytes
Communication time: 0.00003799 seconds
Throughput: 1052908660.17 bytes/second
srikan@DESKTOP-0NIMS2G:~/week4/week5$ nano ls_grep.c
srikan@DESKTOP-0NIMS2G:~/week4/week5$ gcc ls_grep.c -o ls_grep
srikan@DESKTOP-0NIMS2G:~/week4/week5$ ./ls_grep
-rw-r--r-- 1 srikan srikan 1317 Sep 26 09:02 ls_grep.c
-rwxr-xr-x 1 srikan srikan 16408 Sep 26 08:59 producer_consumer
-rw-r--r-- 1 srikan srikan 1762 Sep 26 08:58 producer_consumer.c
Pipeline execution completed.
srikan@DESKTOP-0NIMS2G:~/week4/week5$
```



Observation

- A pipe was created using the `pipe()` system call to establish communication between two child processes.
- The first child used `dup2()` to redirect its standard output to the pipe and executed `ls -l` using `exec()`.
- The second child redirected its standard input from the pipe using `dup2()` and executed `grep .c` using `exec()`.

- The output produced by `ls -l` was therefore passed directly to `grep`, just like a normal shell pipeline.
- The parent process closed the unused pipe ends and used `waitpid()` to synchronize with both child processes.
- The program successfully demonstrated IPC using pipes, file-descriptor redirection using `dup2()`, process creation using `fork()`, and command execution using `exec()`.